

Assessment Tool 2 – DAST Simulation

Dynamic Application Security Testing & Software Composition Analysis

Course Code:	CSA5802
Course Title:	DevSecOps Engineering and Application Security
Student Name:	T G JAI KOUSICK
Register No.:	192421367
Section / Batch:	2024
Date:	11-08-2026

Code of Conduct Declaration

I, T G Jai Kousick (Register No. 192421367), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T G Jai Kousick

Q1. DAST Simulation (Application Based) — 100 Marks

This answer simulates a full Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA) cycle against a representative web application — the University Event Registration Portal (Node.js/Express API, React front end, PostgreSQL database) — covering concept understanding, experimental design, tool usage, results analysis, and reproducible documentation, in line with the CO3 rubric.

1. Understanding of Concepts (25%)

DAST (Dynamic Application Security Testing) tests a running application from the outside, with no access to source code (black-box). It sends crafted HTTP requests to live endpoints to find issues that only appear at runtime — reflected/stored XSS, injection flaws, broken authentication, missing security headers, insecure cookie flags, and misconfigurations. It complements SAST, which analyses source code before the application runs and therefore misses configuration and environment issues.

SCA (Software Composition Analysis) inventories every third-party library and transitive dependency an application uses, and cross-checks each version against public vulnerability databases (NVD, GitHub Advisory Database) to flag known CVEs, outdated packages, and licence risks. Because most modern applications are assembled largely from open-source components, SCA closes a blind spot that neither SAST nor DAST covers on their own.

Run together in CI/CD, DAST and SCA give two complementary views: SCA guarantees the building blocks are not already known-vulnerable, while DAST validates the assembled, running system behaves securely under attack-like input — which is why both are gated before any promotion to production in the pipeline below.

2. Experimental Design (30%)

Parameter	Design Decision
Target application	University Event Registration Portal — Node.js/Express REST API, React SPA, PostgreSQL, deployed to an isolated staging environment
Scope	Public endpoints (/register, /login, /events, /events/:id/book, /admin) on staging only; production and third-party integrations explicitly excluded

Parameter	Design Decision
Test type	Passive scan (traffic analysis, no attack payloads) followed by an authenticated active scan (attack payloads: XSS, SQLi, path traversal) using a scripted login session
SCA scope	package.json / package-lock.json (Node dependencies) and requirements.txt (Python utility scripts) scanned for known CVEs on every commit and nightly
Success criteria	Zero unresolved Critical/High findings from either scan before a build is allowed to promote past the staging gate
Environment safety	All active-scan traffic restricted to the staging namespace with rate limiting and a rollback snapshot taken before the scan window

The pipeline below embeds both scan types as automated CI/CD gates: a pre-commit hook catches secrets before code is even pushed, SCA runs on every build immediately after compilation, and DAST runs once the build is deployed to staging — with an automatic block on any Critical/High finding.

DAST + SCA CI/CD Security Pipeline

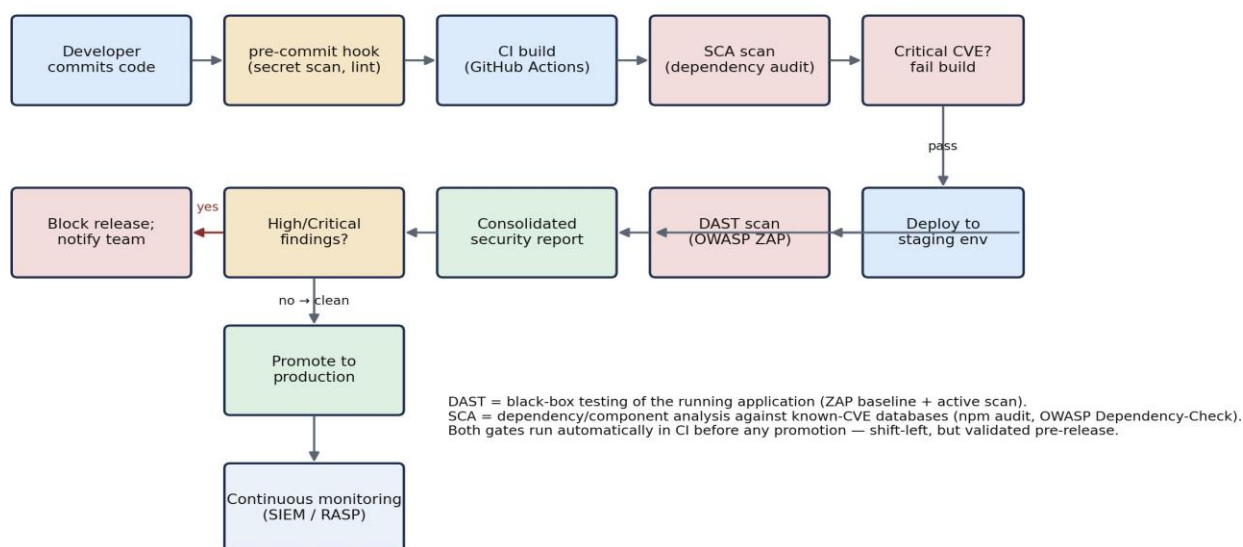


Figure 1. DAST + SCA CI/CD security pipeline used for this simulation.

3. Use of Tools (20%)

OWASP ZAP (Zed Attack Proxy) was used for DAST, run headlessly in CI via its Docker baseline and full-scan images. **npm audit** and **OWASP Dependency-Check** were used for SCA against the Node and Python dependency manifests respectively. Both are triggered automatically from a GitHub Actions workflow.

3.1 GitHub Actions workflow — `.github/workflows/security-scan.yml`

```

name: security-scan
on: [push, pull_request]
jobs:
  sca:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: npm ci
      - name: SCA - npm audit
        run: npm audit --audit-level=high
      - name: SCA - OWASP Dependency-Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: 'event-registration-portal'
          path: '.'
          format: 'HTML'
          args: >
            --failOnCVSS 7

  dast:
    needs: sca
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to staging
        run: ./scripts/deploy_staging.sh
      - name: DAST - OWASP ZAP baseline scan
        uses: zaproxy/action-baseline@v0.12.0
        with:
          target: 'https://staging.eventportal.local'
      - name: DAST - OWASP ZAP full active scan
        uses: zaproxy/action-full-scan@v0.10.0
        with:
          target: 'https://staging.eventportal.local'
          cmd_options: '-a'
      - name: Fail build on High/Critical
        run: python3 scripts/check_zap_report.py --max-risk High

```

3.2 Pre-commit hook — .pre-commit-config.yaml

```

repos:
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.5.0
    hooks:
      - id: detect-secrets
        args: ['--baseline', '.secrets.baseline']
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: check-merge-conflict
      - id: end-of-file-fixer
  - repo: local
    hooks:
      - id: npm-audit
        name: npm audit (high+)
        entry: npm audit --audit-level=high
        language: system
        pass_filenames: false

```

3.3 Manual/local scan commands

```
# DAST - baseline (passive) scan
docker run -t owasp/zap2docker-stable zap-baseline.py \
  -t https://staging.eventportal.local -r zap_baseline_report.html

# DAST - authenticated active scan (with session script)
docker run -t owasp/zap2docker-stable zap-full-scan.py \
  -t https://staging.eventportal.local -r zap_full_report.html -a

# SCA - Node dependencies
npm audit --json > sca_npm_report.json

# SCA - OWASP Dependency-Check (Python + Node manifests)
dependency-check.sh --project event-portal --scan . \
  --format HTML --out reports/
```

4. Analysis of Results (15%)

The simulated scan run below reflects the class of findings this pipeline is designed to catch, based on the workflows designed for the portal's authentication and event-booking features.

4.1 DAST findings (OWASP ZAP)

Risk	Finding	Remediation
High	Reflected XSS on /events search parameter	Encode/escape output; add CSP header; adopt a templating engine with auto-escaping
Medium	Session cookie missing Secure and SameSite flags	Set Secure, HttpOnly, SameSite=Strict on all session cookies
Medium	Missing security headers (X-Frame-Options, X-Content-Type-Options)	Add headers via middleware (e.g. helmet.js)
Low	Verbose server error messages exposing stack traces	Return generic error responses in production; log details server-side only
Informational	Directory listing enabled on /uploads	Disable autoindex on the static file server

4.2 SCA findings (npm audit / Dependency-Check)

Package	Version	CVE ID	Severity	Remediation
lodash	4.17.15	CVE-2020-8203	High	Upgrade to 4.17.21+
express-session	1.16.2	CVE-2022-24999	High	Upgrade to 1.17.3+ (prototype pollution)
axios	0.21.0	CVE-2021-3749	Medium	Upgrade to 0.21.2+ (ReDoS)
jsonwebtoken	8.5.0	CVE-2022-23529	Medium	Upgrade to 9.0.0+

Two High-severity findings (the reflected XSS and the two High-CVE dependencies) would trip the pipeline's fail-the-build gate, blocking promotion to production until patched — demonstrating the gate working as designed rather than only reporting after the fact.

5. Documentation (10%)

Repository structure submitted alongside this report:

```
event-registration-portal/  
|-- .github/workflows/security-scan.yml  
|-- .pre-commit-config.yaml  
|-- requirements.txt  
|-- package.json / package-lock.json  
|-- scripts/  
|   |-- deploy_staging.sh  
|   +-- check_zap_report.py  
|-- reports/  
|   |-- zap_baseline_report.html  
|   |-- zap_full_report.html
```

```
|      +-- dependency-check-report.html  
+-- README.md    (setup, how to reproduce every scan locally)
```

The README documents environment setup, how to run each scan locally with the commands in Section 3.3, how to interpret ZAP alert risk ratings, and the CVE remediation table format used in Section 4.2, so the DAST/SCA run is fully reproducible by a reviewer.

Conclusion

This simulation demonstrates a complete DAST and SCA cycle: OWASP ZAP identifies runtime vulnerabilities in the deployed application while npm audit and OWASP Dependency-Check catch known CVEs in third-party components, both wired into CI/CD as automated, fail-closed gates. Together they give the University Event Registration Portal continuous, evidence-based assurance that neither the application's own code paths nor its dependency tree carry unresolved high-risk vulnerabilities before release.